A Versatile Point Cloud Compressor Using Universal Multiscale Conditional Coding – Part II: Attribute

Jianqiang Wang, Ruixiang Xue, *Graduate Student Member, IEEE*, Jiaxin Li, Dandan Ding, *Senior Member, IEEE*, Yi Lin, and Zhan Ma, *Senior Member, IEEE*

***Abstract*—A universal multiscale conditional coding framework, *Unicorn*, is proposed to code the geometry and attribute of any given point cloud. Attribute compression is discussed in Part II of this paper, while geometry compression is given in Part I of this paper. We first construct the multiscale sparse tensors of each voxelized point cloud attribute frame. Since attribute components exhibit very different intrinsic characteristics from the geometry element, e.g., 8-bit RGB color versus 1-bit occupancy, we process the attribute residual between lower-scale reconstruction and current-scale data. Similarly, we leverage spatially lower-scale priors in the current frame and (previously processed) temporal reference frame to improve the probability estimation of attribute intensity through conditional residual prediction in lossless mode or enhance the attribute reconstruction through progressive residual refinement in lossy mode for better performance. The proposed *Unicorn* is a versatile, learning-based solution capable of compressing a great variety of static and dynamic point clouds in both lossy and lossless modes. Following the same evaluation criteria, *Unicorn* significantly outperforms standard-compliant approaches like MPEG G-PCC, V-PCC, and other learning-based solutions, yielding state-of-the-art compression efficiency with affordable encoding/decoding runtime.**

***Index Terms*—Attribute, conditional coding, geometry, multiscale sparse representation, point cloud compression.**

I. Introduction

PART II of the paper discusses the compression of point cloud attributes, e.g., RGB color intensities of object point clouds in AR/VR (Augmented Reality/Virtual Reality) applications and the reflectance of scene LiDAR scans used by autonomous machines for intelligent tasks. Although the same multiscale conditional coding engine as geometry coding in Part I [1] of the paper is shared to build the high-level modules of attribute coding, it is substantially extended by thoroughly leveraging attribute characteristics for better compression performance.

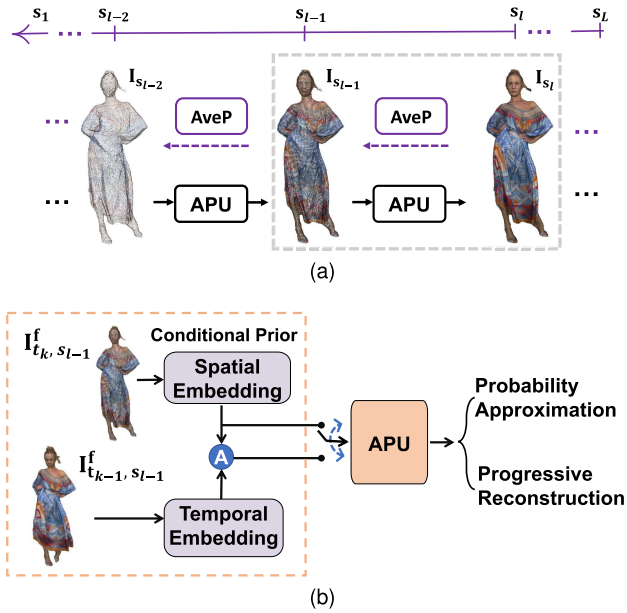


Fig. 1. Attribute coder in *Unicorn*. (a) Multiscale sparse tensors $\{\mathbf{I}_{s_l}\}_{s_1}^{s_L}$ of point cloud attribute are generated and compressed; (b) Spatially or spatiotemporally lower-scale priors are used in APU to support probability approximation in lossless mode or progressive reconstruction in lossy mode for respective static or dynamic coding. "A": feature-space information aggregation.

A. Universal Multiscale Conditional Coding

A dynamic point cloud $\mathcal{P} = \{\mathbf{P}_{t_k}\}_{t_1}^{t_T}, k \in [1, T]$ is a collection of static point cloud frames over time. We separately process the geometry and attribute components of $\mathbf{P}_{t_k}$: the geometry occupancy $\mathbf{O}_{t_k}$ is first compressed, and then the compression of attribute $\mathbf{I}_{t_k}$ is fulfilled upon the decoded geometry $\hat{\mathbf{O}}_{t_k}$.

For a given attribute tensor $\mathbf{I}_{t_k}$, *Unicorn* progressively performs the Average Pooling (AveP) to generate multiscale sparse tensors $\{\mathbf{I}_{t_k,s_l}\}_{s_1}^{s_L}$, $l \in [1, L]$ along with the dyadic geometry downsampling, shown in Fig. 1(a), i.e.,

$$\mathbf{I}_{t_k,s_L} \xrightarrow{\text{AveP}} \mathbf{I}_{t_k,s_{L-1}} \cdots \xrightarrow{\text{AveP}} \mathbf{I}_{t_k,s_l} \cdots \mathbf{I}_{t_k,s_1}. \quad (1)$$

Received 11 December 2023; revised 27 August 2024; accepted 12 September 2024. Date of publication 17 September 2024; date of current version 4 December 2024. This work was supported in part by the Natural Science Foundation of Jiangsu Province under Grant BK20243038, and in part by the National Natural Science Foundation of China under Grant 62171174 and Grant 62331005. Recommended for acceptance by J. Kwon. *(Corresponding authors: Zhan Ma; Yi Lin.)*

Jianqiang Wang, Ruixiang Xue, Jiaxin Li, and Zhan Ma are with the School of Electronic Science and Engineering, Nanjing University, Nanjing, Jiangsu 210093, China (e-mail: wangjq@smail.nju.edu.cn; xrxee@smail.nju.edu.cn; lijiaxin@smail.nju.edu.cn; mazhan@nju.edu.cn).

Dandan Ding is with the School of Information Science and Engineering, Hangzhou Normal University, Hangzhou, Zhejiang 311121, China (e-mail: DandanDing@hznu.edu.cn).

Yi Lin is with the Department of Cardiovascular Surgery of Zhongshan Hospital, Fudan University, Shanghai 200032, China (e-mail: lin.yi@zs-hospital.sh.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TPAMI.2024.3462945, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2024.3462945

Upon generated $\{\mathbf{I}_{t_k,s_l}\}_{s_1}^{s_L}$, point cloud attribute compression (PCAC) starts from its spatially lowest-scale tensor $\mathbf{I}_{t_k,s_1}$ and finally arrives at the highest scale s_L to process $\mathbf{I}_{t_k,s_L}$. An Attribute Processing Unit (APU), as pictured in Fig. 1(b), is devised between any two consecutive scales to leverage lower-scale prior information to compress the current-scale data (e.g., attribute intensity).

Adapting functionalities in APU can easily facilitate the lossless or lossy modes. Similar to the setup in the geometry coder of *Unicorn*, the lossless attribute coder uniformly applies lossless APUs for all scales, and the lossy attribute coder is a two-phase operation, i.e., first devising lossless APUs from the very first scale s_1 to m-th scale s_m, and then employing lossy APUs for all the remaining steps till s_L.

Furthermore, by feeding appropriate prior information to APU, i.e., spatially or spatiotemporally, we can flexibly support the *static* or *dynamic* coding of the point cloud attribute frame.

Furthermore, devising a simple and basic neural network (e.g., using sparse convolution) is sufficient to fulfill APU's functions in the proposed framework.

B. From Geometry Coding to Attribute Coding

Though similar multiscale processing is applied for geometry and attribute compression, the intrinsic characteristics of point cloud attributes are carefully considered when designing APU.

As attribute intensity exhibits a much wider dynamic range than geometry occupancy, e.g., 8-bit vs. 1-bit, directly predicting the intensity level of each attribute element is practically inefficient. For instance, an 8-bit reflectance element has $2^8 = 256$ possible levels to predict. In contrast, geometry occupancy only has a binary status, whose estimation is much easier.

Upon the multiscale representation structure in Fig. 1(a), assuming we want to compress the data at scale s_l, i.e., $\mathbf{I}_{s_l}$, we have lower-scale prior $\mathbf{I}^f_{s_{l-1}} = \mathbf{I}_{s_{l-1}}$ in lossless mode, while in lossy mode, $\mathbf{I}^f_{s_{l-1}} = \hat{\mathbf{I}}_{s_{l-1}}$ with compression noise. Other casual knowledge, spatially or spatiotemporally, can also be fused with $\mathbf{I}^f_{s_{l-1}}$ if necessary for better performance.

Subsequently, we propose approximating the attribute residual between the current scale and the preceding lower scale. Since $\mathbf{I}_{s_{l-1}}$ is generated from $\mathbf{I}_{s_l}$ via average pooling as in (1), its compressed reconstruction still closely correlates with $\mathbf{I}_{s_l}$. As a result, the corresponding attribute residual between them shall present a more compact distribution, which can be effectively characterized.

- As for lossless mode, we directly model the distribution of attribute residual using a Laplacian function, for which we can deduce the probability of compressing the attribute itself; on the other hand,
- As for lossy mode, we encode transform-domain attribute residual using a context-adaptive engine conditioned on the lower-scale prior, by which we progressively apply the residual refinement from one lower scale to a higher one recurrently.

Recalling that we process the geometry and attribute parts in *Unicorn* separately and the geometry occupancy is first compressed, we thus have the full knowledge of occupied voxels, e.g., total quantity and specific coordinates. As a result, we propose geometry-aware processing to improve the efficiency. Note that attribute unpooling is devised from s_{l-1} to s_l along with the dyadic geometry upsampling, where one occupied voxel at s_{l-1} is expanded to a $2 \times 2 \times 2$ cube with eight nodes (a.k.a., octree expansion).

- *Gometry-aware Skipping (GaS)* is motivated to speed up the computation. If only one occupied voxel exists in a $2 \times 2 \times 2$ cube at scale s_l given the geometry prior condition, we can directly infer its attribute using the attribute of that occupied voxel at scale s_{l-1}.
- *Geometry-aware Updating (GaU)* is applied in the lossless mode for conditional probability estimation of grouped attribute elements in a multistage means. Along with the group-wise processing, we can update the prediction of upcoming grouped elements using attribute levels in previously processed groups, which turns out to produce more precise probability estimation (see Fig. 2(c)).

C. Contributions

The proposed *Unicorn*, for the very first time, unifies the compression of the geometry occupancy and attribute intensity of any input point cloud in the lossy or lossless mode of either static or dynamic coding scenarios under a universal multiscale conditional coding framework. On the contrary, existing methods mostly focus on a specific scenario. Even for the MPEG G-PCC standard, it is not a strictly unified solution since it applies different processing trees to compress geometry and attribute information, not to mention that dedicated tools have also been engineered to tackle specific occasions (e.g., isolated points).

1) Upon the multiscale representation framework, *Unicorn*'s attribute coder processes the attribute residual between consecutive scales, offering a coarse-to-fine representation through progressive residual refinement. Such a progressive decoding/reconstruction capacity is practically valuable for networked streaming applications with bandwidth fluctuation.
2) When only concerning the attribute compression efficiency assuming losslessly compressed geometry, *Unicorn* provides notable gains to state-of-the-art G-PCC, i.e., $>20\%$ BD-BR gains in lossy coding, and it also significantly surpasses existing learning-based solutions. When considering the scenario compressing the geometry and attribute jointly in lossy mode, *Unicorn* reveals more than 40% and 25% bitrate reductions, respectively, to G-PCC and V-PCC at the same overall quality measured using the prevalent PCQM metric [2].
3) The attribute coder in *Unicorn* is a low-complexity approach, exhibiting a similar encoding/decoding runtime as G-PCC at the second level, which shows the speed-up of more than two orders of magnitude against recently-emerged SparsePCAC [3], ScalablePCAC [4], and GDL-PCAC [5]. It also uses a single neural model to support variable-rate coding. These promise encouraging prospects for *Unicorn* in practical applications.

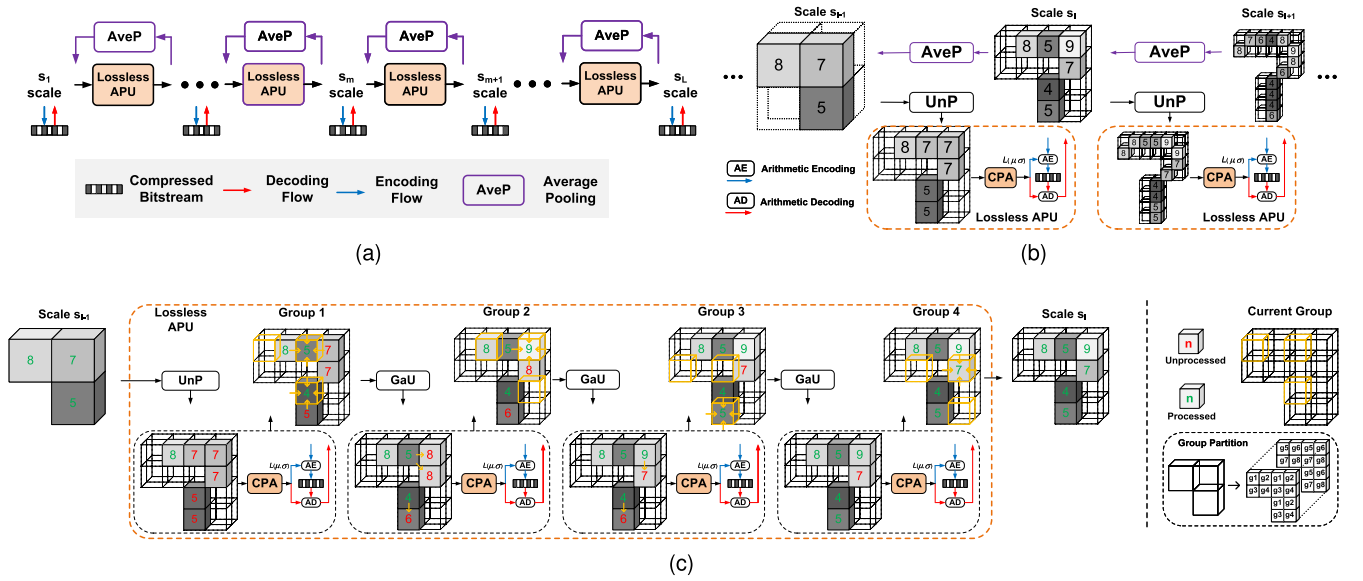


Fig. 2. Lossless attribute coder. (a) Lossless APUs are uniformly devised for Conditional Probability Approximation (CPA) in compression across all scales. (b) One-stage CPA is exemplified. The UnP (UnPooling) block upsamples the lower-scale reconstruction as the initial prediction of current-scale attributes for CPA; (c) Multistage CPA is fulfilled by stacking multiple CPA to process grouped elements sequentially, in which the GaU (Geometry-aware Updating) is applied to update the initial prediction using geometry prior and previously-processed same-scale neighbors for more accurate CPA. Here, single-channel attributes are illustrated with the intensity levels annotated.

TABLE I
NOTATIONS

Notation	Description
PCAC	Point Cloud Attribute Compression
APU	Attribute Processing Unit
CPA	Conditional Probability Approximation
FeCPA	Feature-space CPA
AveP	Average Pooling
UnP	UnPooling
$\mathcal{P}=\{\mathbf{P}_{t_k}\}_{t_1}^{t_T}$	Dynamic point cloud $\mathcal{P}$ with a serial static point cloud frames $\mathbf{P}_{t_k}$
t_k, s_l	time index t_k, spatial scale s_l
$\mathbf{P}_{t_k,s_l}$	Point cloud tensor at t_k and s_l
$\mathbf{I}_{t_k,s_l}$	Attribute intensity tensor of $\mathbf{P}_{t_k,s_l}$
$\mathbf{I}^f_{t_k,s_{l-1}}$	Lower-scale prior of $\mathbf{I}_{t_k,s_l}$

Table I lists frequently used notations in this work.

II. Related Works

A general review of point cloud compression was given in Part I of this paper [1]. This section thus discusses attribute compression, particularly emphasizing learned solutions.

A. Traditional PCAC Methods

Similarly, PCAC studies rely on proper organizational structures to exploit correlations for attribute compression.

Predictive Tree was one direction extensively explored in the past. To the best of our knowledge, the earliest work along this avenue was the binary predictive tree proposed in [6]. Shortly, the octree-structured attribute compression was discussed in [7], [8]. Lately, dedicated transforms have been developed to better exploit attribute correlations.

For example, Graph Fourier Transform (GFT) and its variants were examined in [9], [10], [11], [12], [13]. However, these GFTs are too complex when performing the graph Laplacian eigendecomposition. To alleviate the computational burden, Queiroz et al. [14] developed the Region-Adaptive Hierarchical Transform (RAHT), an adaptive Haar wavelet transform, which was later adopted in the MPEG G-PCC reference model. In the meantime, G-PCC also adopted Predicting/Lifting Transforms to compress the attribute according to the organization of Level of Details (LoDs).

3D-to-2D Projection offered an alternative solution to reuse prevalent 2D image/video coders to compress projected or mapped 2D image grids or planes. A representative work is MPEG V-PCC [15], which applies patch-based projection to form attribute images for compression.

An excellent review in this regard was given in [16].

B. AI-Based PCAC Methods

Recently, applying deep learning to improve attribute compression attracted extensive attention.

End-to-end solutions typically devise autoencoders to exploit 3D spatial correlations for the input source's compact representation. For example, upon the 3D uniform voxel model, Alexiou et al. [17] directly applied 3D dense convolutions to compress color attributes of the input point cloud. Shortly, Sheng et al. proposed using point convolutions and point-inception blocks to form a point model-based method, dubbed Deep-PCAC [18] for the same purpose. Both attempts in [17] and [18] had to slice the input attribute tensor into small patches due to complexity concerns, and their lossy compression efficiency

still largely underperformed to G-PCC's early test model version (i.e., TMC13v10).

Using sparse tensor representation, SparsePCAC [3] simply stacked sparse convolutions to lossily compress point cloud attributes. Although it reported noticeable improvements to Deep-PCAC (e.g., better performance and less complexity), its performance was still inferior to the G-PCC (TMC13v14) and later releases.

A recent attempt using graph dictionary learning was reported in [5], which utilized ρ-Laplacian embedding to regularize dictionary atoms to fit underlying geometric structures for compact representation of point cloud attribute. Average 6% to 8% BD-BR gains were achieved to G-PCC (TMC13v14) when testing on MPEG object point clouds.

Previous discussions focus on the lossy coding of point cloud attributes. In addition, Nguyen et al. proposed the deep entropy models for end-to-end lossless attribute coding, named CNeT [19] and MNeT [20]. In CNeT, they used an autoregressive context model to predict the probability distributions of color attribute, while in MNeT, they devised a multiscale framework to project attribute onto multiscale latent spaces for acceleration.

Neural Post-processing offers an alternative direction to restore G-PCC compressed point cloud attribute for a better reconstruction.

For example, G-PCC compressed color attribute was improved by quality improvement post-processing filters like graph-based MS-GAT [21] and sparse tensor-based ARNet [22]. And, ARNet showed superior performance to MS-GAT with much less complexity (e.g., about 30× speedup). Lately, a lightweight G-PCC++ model [23] extended the study to restore G-PCC compressed geometry and attribute components jointly rather than merely dealing with the color attribute as in ARNet and MS-GAT.

Layered Enhancement usually devises learned models to refine G-PCC compressed point clouds, for which the standard G-PCC is employed at the base layer to encode the thumbnail point cloud downscaled from the original input, and neural models are applied at the enhancement layer to compress the original input conditioned on the base layer prior for better reconstruction.

ScalablePCAC [4] is an example in this category for refining G-PCC compressed color attributes. ScalablePCAC demonstrated >10% BD-BR reduction against G-PCC (TMC13v22) on common test sequences.

More or less the same time, YOGA [24] extended ScalablePCAC by refining both geometry and attribute components conditioned on the base layer G-PCC reconstruction. The neural model used in YOGA's enhancement layer was unified for compressing geometry and attribute information, which was extended from [25].

Alternative PCAC methods using 3D-to-2D projection, Implicit Neural Representation (INR), etc., are also under extensive investigation. For instance, Quach et al. [26] folded 3D point cloud attribute onto the 2D grids to form a mapped 2D color image and then applied HEVC (High-Efficiency Video Coding) standard compliant intra-coding profile [27] to encode it. Isik et al. [28] used coordinate-based networks, e.g., LVAC, to infer point cloud attribute. When using such an INR-based method, the attribute compression was transformed as the compression of the underlying neural network. Unfortunately, both projection-based and INR-based solutions are still inferior to G-PCC.

Besides, Fang et al. [29] proposed using a learned entropy model to replace G-PCC's handcrafted contextual rules, *a.k.a.* 3DAC, for which they wished to better capture the probability distribution of RAHT coefficients for encoding. Gains were observed over G-PCC (TMC13v13) on samples from ScanNet and SemanticKITTI datasets but not on common test samples widely adopted in the standardization committee.

III. Attribute Compression Using *Unicorn*

To quickly digest the proposed *Unicorn*, we start the discussion of lossless attribute compression in Section III-B and then extend it to the lossy mode in Section III-C, for static coding using spatial prior only. Dynamic coding is subsequently exemplified by allowing spatiotemporal priors in Section III-D.

The aforementioned studies assume losslessly compressed geometry to only evaluate the efficiency of attribute compression. Targeting popular point cloud applications like telepresence or holographic streaming across the Internet, geometry and attribute components must be both lossily compressed as discussed in Section III-E.

A. Multiscale Attribute Tensor Generation via Average Pooling (AveP)

Figs. 2(a), 3(a), and 4 illustrate the multiscale conditional coder supported by the proposed *Unicorn*. To compress the original input attribute tensor $\mathbf{I}_{s_L}$, the first step is to generate multiscale attribute tensors.

Given an attribute tensor $\mathbf{I}_{s_l} = \{x_i^{(s_l)}, 1 \le i \le N^{(s_l)}\}$ at scale s_l, corresponding k occupied voxels in a j-th $2 \times 2 \times 2$ local cube are merged into an occupied voxel at the lower scale s_{l-1}, having

$$x_j^{(s_{l-1})} = \left\lfloor \frac{1}{k} \sum_{m=1}^{k} x_m^{(s_l)} \right\rceil, k \in \{1, 2, \ldots, 8\}. \tag{2}$$

$\lfloor \cdot \rceil$ is the rounding operator and $\mathbf{I}_{s_{l-1}} = \{x_j^{(s_{l-1})}, 1 \le j \le N^{(s_{l-1})}\}$. $N^{(s_l)}$ is the total number of occupied voxels at scale s_l, and $x_i^{(s_l)}$ is the attribute intensity for i-th occupied voxel at scale s_l, which is an integer in general. The same AveP is applied along with the dyadic geometry downsampling to generate multiscale sparse tensors $\{\mathbf{I}_{s_l}\}_{s_1}^{s_L}$ of the original input $\mathbf{I}_{s_L}$. Fig. 2(b) intuitively exemplifies AveP from scale s_{l+1} to s_{l-1}.

Similarly, the compression process starts from the lowest scale s_1 to the highest scale s_L. When compressing $\mathbf{I}_{s_l}$, its lower-scale reconstruction is known and utilized as the conditional prior knowledge to assist the compression of $\mathbf{I}_{s_l}$.

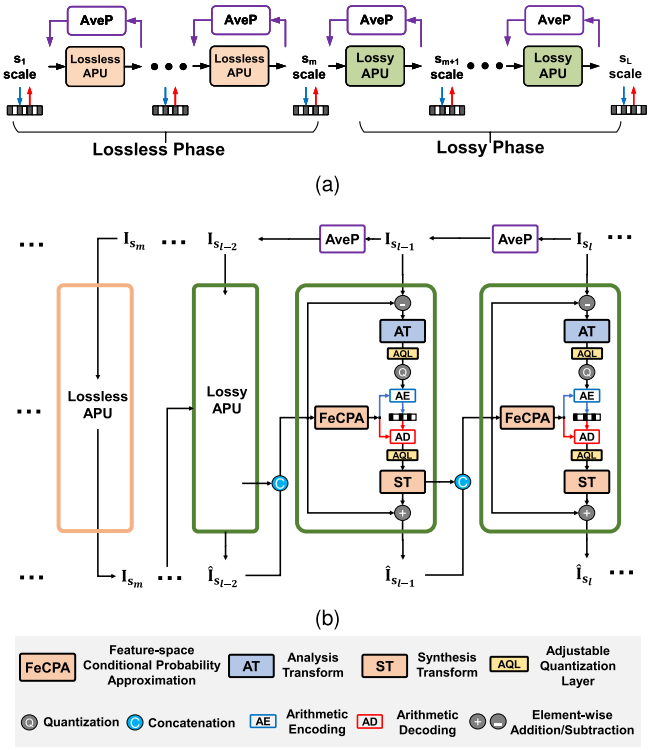


Fig. 3. Lossy attribute coder. (a) Multiscale representation framework with a two-phase setup; (b) Lossy APU adopts Transform-domain conditional residual coding to progressively refine the attribute reconstructions scale by scale.

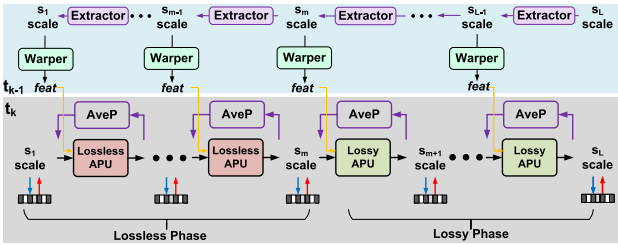


Fig. 4. Lossy dynamic attribute coder. Multiscale temporal priors are generated using Extractor, transferred using Warper, and concatenated with the spatial prior for the processing in lossless and lossy APUs.

B. Lossless Attribute Compression via Conditional Probability Approximation (CPA)

UnP. To compress $\mathbf{I}_{s_l}$ at scale s_l, we have its lower-scale reconstruction $\mathbf{I}_{s_{l-1}}$[1] available as the conditional prior $\mathbf{I}^f_{s_{l-1}}$. As for $\mathbf{I}_{s_{l-1}}$, its j-th occupied voxel $x_j^{(s_{l-1})}$, is first geometrically expanded to a local $2\times2\times2$ cube j with eight child nodes (*a.k.a.*, octree expansion). Since the geometry occupancy is available in advance when processing attribute components of a point cloud, k known occupied voxels in this cube j is filled with the same $x_j^{(s_{l-1})}$, e.g., $\tilde{x}_m^{(s_l)} = x_j^{(s_{l-1})}$, $m \in [1,k]$.

[1]We use $\hat{\mathbf{I}}_{s_{l-1}}$ to represent the reconstruction to make a differentiation from the corresponding uncompressed input $\mathbf{I}_{s_{l-1}}$. Here, as for lossless coding, $\hat{\mathbf{I}}_{s_{l-1}} = \mathbf{I}_{s_{l-1}}$.

Such a step is referred to as the UnPooling (UnP) process, which is iterated for all elements in $\mathbf{I}_{s_{l-1}}$ to produce a tensor as the initial prediction of $\mathbf{I}_{s_l}$, i.e.,

$$\mathbf{I}^{init}_{s_l} = \{\tilde{x}_i^{(s_l)}\} = \mathbf{UnP}(\mathbf{I}_{s_{l-1}}). \tag{3}$$

CPA: Subsequently, the Conditional Probability Approximation (CPA) model inputs $\mathbf{I}^{init}_{s_l}$ to derive the probability for each element in $\mathbf{I}_{s_l}$, e.g.,

$$p(\mathbf{I}_{s_l}|\mathbf{I}^f_{s_{l-1}}) = \mathbf{CPA}(\mathbf{I}^{init}_{s_l}). \tag{4}$$

The processing of (4) comprises two steps:

1) Generating means $\{\mu_i\}$ and variances $\{\sigma_i\}$ for all elements $\{x_i^{(s_l)}\} \in \mathbf{I}_{s_l}$ assuming the Laplacian distributed probability density function $\mathcal{L}$;
2) Deducing the probability by integrating $\mathcal{L}$ via

$$\prod\nolimits_i \big(\mathcal{L}(\mu_i,\sigma_i) * \mathcal{U}(-\tfrac{1}{2},\tfrac{1}{2})\big)(x_i^{(s_l)}), \tag{5}$$

to encode $x_i^{(s_l)}$ into the bitstream; or decoding a level from the bitstream to reverse (5) for reconstructing $x_i^{(s_l)}$. $\mathcal{U}(-\frac{1}{2},\frac{1}{2})$ is the uniform distribution ranging from $-\frac{1}{2}$ to $\frac{1}{2}$.

The proposed CPA in (4) can be easily facilitated using a DNN model. As the UnP process in (3) offers an initial approximation of current-scale attribute intensity, we, in practice, predict the difference between $\mathbf{I}_{s_l}$ and $\mathbf{I}^{init}_{s_l}$, which can be easily implemented by adding a residual connection in CPA's neural model.

Multistage CPA. CPA in (4) performs one-shot estimation for all elements in $\mathbf{I}_{s_l}$. It can be extended to process elements from one group to another in a multistage manner, by which same-scale and nearby neighboring elements in previously processed groups can be further utilized to exploit correlations better. It is dubbed multistage CPA.

Here, we give an example of using multistage CPA to process $\mathbf{I}_{s_l}$ with eight groups in Fig. 2(c). Specifically, $\mathbf{I}_{s_l}$ is organized as a collection of $2\times2\times2$ cubes, each of which contains eight inter-connected voxels (e.g., some of them are occupied voxels and the remaining ones are unoccupied voxels). For each $2\times2\times2$ cube, we mark the voxel with its group identification according to the geometric position in a raster scanning order (see "g1" to "g8" in the legend plot). As a result, a total of eight groups are specified, having $\mathbf{I}_{s_l} = \{\mathbf{I}_{s_l,g_i}\}_{g_1}^{g_8}$.

Instead of making a one-shot probability estimation for all elements in $\mathbf{I}_{s_l}$ as in (4), CPA is applied in a group-wise fashion, where both lower-scale reconstruction $\mathbf{I}_{s_{l-1}}$ (after the UnP process) and previously processed elements in preceding groups, e.g., $\{\mathbf{I}_{s_l,g_{<i}}\}$ are employed as the spatial priors for conditional prediction. Here, we simply use $\{\mathbf{I}_{s_l,g_{<i}}\}$ to include all elements before i-th group. Obviously, $p(\mathbf{I}_{s_l,g_i}|\mathbf{I}^f_{s_{l-1}}, \{\mathbf{I}_{s_l,g_{<i}}\}) > p(\mathbf{I}_{s_l,g_i}|\mathbf{I}^f_{s_{l-1}})$, leading to a better compression performance.

As the occupied voxels of each $2\times2\times2$ cube in $\mathbf{I}_{s_l}$ are known in advance, we can process and update $\mathbf{I}^{init}_{s_l,g_i}$ from one group to another before feeding it to CPA, which is so-called as Geometry-aware Updating (GaU), i.e.

$$\mathbf{I}^{init}_{s_l,g_i} = \frac{k\times\mathbf{I}_{s_{l-1}} - \sum\{\mathbf{I}_{s_l,g_{<i}}\}}{k-i}. \tag{6}$$

$k \in \{1, 2, \ldots, 8\}$ is the number of occupied voxels in each $2 \times 2 \times 2$ cube and $i < k$.

When only one occupied voxel exists in a $2 \times 2 \times 2$ cube, we can directly infer its ground-truth value without further steps, which is referred to as GaS (Geometry-aware Skipping). For example, the intensity level 8 of upper-left occupied voxel at scale s_{l-1} directly infers the attribute of the corresponding solely occupied voxel in upscaled $2 \times 2 \times 2$ cube at scale s_l, as shown in Fig. 2(c)

Remark: Previous examinations assume the processing of a single-channel attribute component, which can be directly used for single-channel reflectances of a LiDAR point cloud. For a colorized object point cloud with three-channel RGB colors, we often convert its color space from RGB to YCoCg for lossless coding. Using YCoCg can remove cross-color correlation among R, G, and B components to some extent, which, thus, benefits the compression.[2] Ideally, we can first compress the Y attribute, then Co with the help of Y and Cg with the help of both Y and Co. In our earlier exploration [31], we simultaneously process Co and Cg components conditioning on the Y prior, by which the processing speed can be somewhat accelerated.

C. Lossy Attribute Compression

Fig. 3(a) illustrates the lossy PCAC solution, where it applies lossless APU from scale s_1 to s_m and devises lossy APU thereafter till s_L.

Recalling the studies on feature-space conditional probability approximation in lossy geometry coding, we propose transform-domain conditional coding of attribute residual to facilitate the lossy APU uniformly across all lossy scales. Transforms were extensively studied to represent the point cloud attribute. Notable examples included GFT [9] and RAHT [14], where RAHT was adopted into the G-PCC standard. Recently, learning-driven (nonlinear) neural transforms [3], [4], [5], [18], [32] were introduced into PCAC solutions and demonstrated encouraging efficiency.

Attribute Residual Generation: For a given scale s_l, its attribute tensor is $\mathbf{I}_{s_l}$. The corresponding attribute residual $\mathbf{r}_{s_l}$ is derived via

$$\mathbf{r}_{s_l} = \mathbf{I}_{s_l} - \mathbf{I}^p_{s_l}, \quad \mathbf{I}^p_{s_l} = \mathbf{UnP}(\hat{\mathbf{I}}_{s_{l-1}}). \tag{7}$$

$\mathbf{I}^p_{s_l}$ is the cross-scale prediction generated by the lower-scale reconstruction $\hat{\mathbf{I}}_{s_{l-1}}$. Here, we assume the quantization noise augmentation to differentiate the reconstruction tensor $\hat{\mathbf{I}}_{s_{l-1}}$ from its uncompressed counterpart $\mathbf{I}_{s_{l-1}}$ in lossy mode. It is also worth pointing out that the prediction generation simply applies the UnP process discussed above.

Transform & Quantization: Then, attribute residual $\mathbf{r}_{s_l}$ is transformed and quantized, e.g.,

$$\mathbf{f}^r_{s_l} = \mathrm{Q}(\mathbf{AQL}(\mathbf{AT}(\mathbf{r}_{s_l}))), \quad \hat{\mathbf{r}}_{s_l} = \mathbf{ST}(\mathbf{AQL}(\mathbf{f}^r_{s_l})). \tag{8}$$

[2]Transforming RGB to YUV can also reduce the cross-color redundancy, but it is not mathematically lossless. For lossy attribute compression, we use YUV color space [30].

Following our preliminary studies in SparsePCAC [3], we stack (sparse) convolutional layers to form the Analysis Transform (AT) and Synthesis Transform (ST) under an autoencoder structure. As many existing works focus on rate-specific models [3], [4], we introduce Adjustable Quantization Layer (AQL) ($\mathbf{AQL}(\cdot)$ in encoding and decoding processes shown in Fig. 3(b)) for variable-rate coding using a single neural model, which significantly reduces the complexity for potential applications in practice. Such an AQL mechanism usually scales latent elements in $\mathbf{f}^r_{s_l}$ in encoding and decoding to fulfill the purpose, and the scaling can be implemented using either numerical factors or network models [33], [34]. More details can be found in the supplemental material.

FeCPA: To perform the entropy coding of $\mathbf{f}^r_{s_l}$, we must construct proper contexts by which we can notably reduce the bitrate consumption [35]. Here, lower-scale priors are utilized as the conditional knowledge for probability modeling, e.g.,

$$p(\mathbf{f}^r_{s_l} | \hat{\mathbf{r}}_{s_{l-1}}, \hat{\mathbf{I}}_{s_{l-1}}) = \mathbf{FeCPA}(\mathcal{C}\{\hat{\mathbf{r}}_{s_{l-1}}, \hat{\mathbf{I}}_{s_{l-1}}\}). \tag{9}$$

$\hat{\mathbf{r}}_{s_{l-1}}$ and $\hat{\mathbf{I}}_{s_{l-1}}$ stand for lower-scale reconstructions of attribute residual and attribute intensity, respectively. They are concatenated and fed into the FeCPA unit. Although it is possible only using the closest lower-scale $\hat{\mathbf{r}}_{s_{l-1}}$ to model current-scale residual $\mathbf{f}^r_{s_l}$ in latent space, having $\hat{\mathbf{I}}_{s_{l-1}}$ can provide more conditional knowledge, because $\hat{\mathbf{I}}_{s_{l-1}}$ recurrently aggregates the residual information from all preceding scales through

$$\hat{\mathbf{I}}_{s_{l-1}} = \hat{\mathbf{r}}_{s_{l-1}} + \mathbf{UnP}(\hat{\mathbf{I}}_{s_{l-2}}). \tag{10}$$

Note that the processing steps in (9) are more or less the same as those in (4). The difference lies in element type: in lossless mode, we estimate the probability of attribute intensity itself, while in the lossy mode, the estimation is conducted for transform-domain attribute residual in the latent feature space. Thus, we call (9) Feature-space CPA (FeCPA).

Remark: The lossy attribute coder progressively refines the reconstruction scale by scale, offering a coarse-to-fine representation through the transform-domain conditional coding of attribute residual. Theoretically, it might be possible to conditionally compress the attribute intensity rather than the attribute residual in this work. However, according to our extensive studies, the model used for conditionally compressing the attribute residual is more robust and converges much faster. We believe this is because the attribute residual exhibits a more clustered distribution that is much easier to characterize. Also, the multiscale structure is vital, and enables the proposed method to achieve significant compression gains to state-of-the-art G-PCC codec.

In lossless mode, GaU and GaS are applied in multistage CPA to infer and update attribute intensity without requiring the extra bitstream signaling (see Fig. 2(c)). Here, we apply GaS in lossy mode to leverage the geometry prior. Assuming only one voxel is occupied in a $2 \times 2 \times 2$ cube at scale s_l, we will directly infer it from its octree parent at s_{l-1}.

In lossy mode, RGB colors are transformed to YUV space for compression. Unlike the lossless APU in which individual

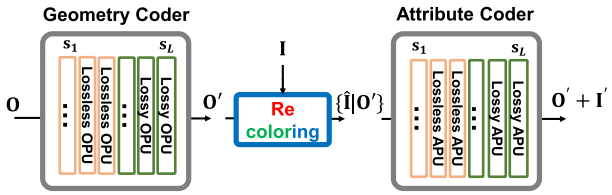


Fig. 5. Unified compression of geometry and attribute in *Unicorn*.

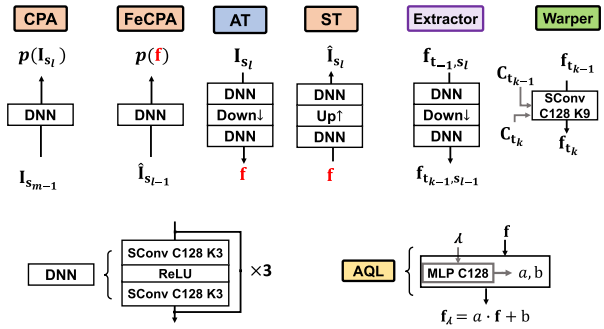


Fig. 6. Neural network units for functional models.

color component is processed separately, lossy APU compresses three-channel YUV together.

D. Dynamic Attribute Coding

Previous discussions assume the *static attribute coding*, for which each point cloud frame is compressed independently, and we can only leverage the spatial prior in the current frame. On the contrary, enabling *dynamic attribute coding* allows us to exploit correlations from both the current and temporal reference frames for better compression.

In Fig. 4, we propose Extractor to progressively aggregate and embed spatial features from s_L to s_1 to formulate multi-scale temporal priors $\{\mathbf{f}^{\mathbf{I}}_{t_{k-1},s_l}\}^{s_L}_{s_1}$ from a temporal point cloud reference $\hat{\mathbf{I}}_{t_{k-1}}$. These priors are attached with the corresponding coordinates since geometry information is known in advance.

To alleviate the impact of temporal motion across consecutive frames, we warp scale-wise temporal priors to the current frame for conditional coding. To this end, Warper is devised to apply a target convolution to transfer the reference's attribute features to the current frame so that the spatial and temporal priors are concatenated and fed into lossy or lossless APU for probability approximation.

The proposed Extractor and Warper modules are implemented by stacking neural network layers, as shown in Fig. 6. A target convolutional layer with a fixed kernel size of $9 \times 9 \times 9$ is utilized in Warper to aggregate spatiotemporal prior for dynamic coding.

E. A Unified Framework for Joint Compression of Geometry & Attribute

Previous sections assume losslessly compressed point cloud geometry when evaluating the efficiency of attribute coding. To sustain networked point cloud applications like holographic communication, both geometry and attribute parts must be compressed in lossy mode to save more bandwidth. Therefore, this section discusses a unified framework to support the lossy coding of geometry and attribute information of any given point cloud.

It is worth pointing out that the separate processing of geometry $\mathbf{O}$ and attribute information $\mathbf{I}$ in *Unicorn* allows us to engineer a unified framework to fulfill the purpose above conveniently. To this end, we must re-map the original attribute tensor $\mathbf{I}$ to the reconstructed geometry $\mathbf{O}'$ before feeding it into the attribute coder. Here, the output of geometry coder $\mathbf{O}'$ is generally assumed with compression noise. Such a process is referred to as "re-coloring".

Re-coloring: For a point $\vec{p}$ in $\mathbf{O}'$, we locate its n nearest neighbors in uncompressed ground-truth $\mathbf{O}$ and compute the distance-weighted average of their attributes as the attribute intensity of $\vec{p}$. Recoloring is finished when we traverse all points in $\mathbf{O}'$. According to our extensive simulation, $n = 3$ minimizes the recoloring-induced distortion. A similar recoloring algorithm is used in G-PCC [30] as well.

We finally have a re-colored attribute tensor $\mathbf{I}'$. In compression, $\mathbf{I}'$ is conditioned on lossily-compressed $\mathbf{O}'$ instead of the original input $\mathbf{O}$ for subsequent computations.

Fig. 5 sketches a unified compressor using multiscale geometry and attribute coders proposed in *Unicorn*. Theoretically, we can flexibly pair a geometry coder with an attribute coder in existing approaches without constraints. For example, we can use the G-PCC octree coder and *Unicorn*'s attribute coder, and we can also use *Unicorn*'s geometry coder and SparsePCAC. Such flexibility mainly owes to the separable processing of geometry and attribute. The proper pairing of geometry and attribute coders for joint compression is justified by the trade-off between performance and complexity from the underlying applications.

F. Functional Model & Network Implementation

Functional models, including CPA and FeCPA, power the attribute coder in *Unicorn*. Other modules, like AT, ST, Extractor, and Warper, are integrated with functional models for compression.

Fig. 6 gives neural network blocks in our implementation.

1) A DNN block stacks three ResNet blocks, mainly comprising sparse convolutions (SConv) using 128 channels and 3×3×3 kernel size, e.g., C128, K3.
   - Such a DNN block is used in CPA to estimate the probability of element intensity in attribute space and is applied in FeCPA to estimate the probability of transform-domain attribute residual in latent space.
   - It is also used in AT, ST, and Extractor units to aggregate and embed neighborhood characteristics as latent features.
2) Down↓ is a sparse convolution layer ($stride = 2^3, kernel\_size = 2^3$), and it is typically integrated with DNN blocks to build up AT or Extractor unit.

TABLE II
DATASETS USED IN STUDY

Categories	TestData	TrainData	Geometry precision	Attributes 8-bit
Static Objects	8iVFB	RWTT	10-bit	RGB
	Owlii		10-bit	RGB
	CTC Samples		11,12-bit	RGB
Static Scenes	ScanNet	ScanNet	2cm	RGB
	KITTI	KITTI	1mm	Reflectance
	Ford	Ford	1mm	Reflectance
Dynamic	soldier	8iVFB+Owlii	10bit	RGB
	basketball		10-bit	RGB

† The split of training and testing samples strictly follows the common conditions [36].

3) Up↑ is a transposed sparse convolution layer ($stride = 2^3, kernel\_size = 2^3$), and it is applied in the ST unit with DNN blocks to upscale latent features.
4) AQL uses a simple MLP (MultiLayer Perceptron) structure to properly scale latent features for variable-rate coding.

As we do not observe performance gains when using the neighborhood self-attention (e.g., NPFormer in Part I [1]) to build up the neural network units, we mainly rely on sparse convolutions in this study for simplicity.

IV. EXPERIMENTAL SETUP

A. Datasets

Table II lists object and scene point cloud datasets used for evaluation. Samples in these datasets exhibit diverse geometry and attribute characteristics. Fig. 7 visualizes some of them for intuitive understanding.

1) Static Coding of Object Point Clouds: Object point clouds with fine geometry textures and vibrant color attributes are commonly used in immersive media applications like AR/VR.

Testing Dataset: Representative 8i Voxelized Full Bodies (8iVFB) and Owlii human mesh (Owlii) are selected as the main test sets for evaluation, and they are recommended by standardization committees and widely used in many existing works [3], [4], [5], [19]. To further demonstrate the effectiveness of the proposed method on various object point clouds, more samples from MPEG CTC (Common Test Conditions)[37] are tested.

- **8iVFB** [38] is one of the most popular point cloud compression testing set used in MPEG. It includes 4 sequences, known as *longdress*, *loot*, *redandblack*, and *soldier*. Each sequence has 300 frames of voxelized human body point clouds with a geometry precision of 10 bits. For static attribute compression, a specific frame from each sequence above, i.e., #1300, #1550, #1200, and #0690, is respectively extracted (i.e., four frames in total for 8iVFB).
- **Owlii** [39] is another popular testing set, consisting of 4 sequences of human body point cloud, i.e., *basketball_player*, *dancer*, *model*, and *exercise*, each of which has 300 frames with 10-bit geometry precision. Frames with indices of #0200, #0001, #0001, and #0001 are extracted from those four sequences respectively for static coding.

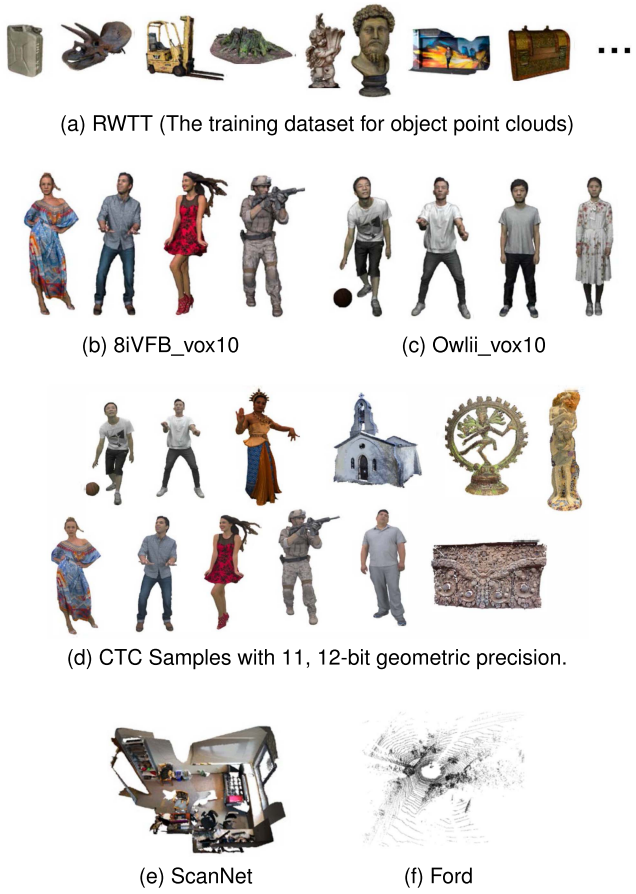


Fig. 7. Visualization of object and scene point clouds used in this work.

- **MPEG CTC Samples** [37] include twelve point clouds. They are acquired using different devices with higher geometric precision (e.g., 11-bit and 12-bit), various geometric densities, and textures, making the compression more challenging. Specific testing samples are 11-bit *basketball_player* and *dancer*; as well as 12-bit *thaidancer*, *soldier*, *loot*, *longdress*, *redandblack*, *Facade*, *House_w/o_roof*, *Shiva*, and *Staue_Klimt*.

Training Dataset: We use the RWTT dataset [40] for training, which comprises 568 3D mesh models with fine-grained textures and vivid colors. These models are generated carefully for real-world 3D reconstruction applications. We densely sampled these mesh models to generate the corresponding point clouds for training.

2) Static Coding of Scene Point Clouds: Scene point clouds are used to represent large-scale 3D indoor or outdoor scenes, which are widely used in machine vision tasks like semantic segmentation, instance detection, etc. We select several well-known datasets, e.g., ScanNet, KITTI, and Ford, for the study in this work. These scene datasets all have thousands of frames, we follow the common dataset split rule suggested in [36] to perform training and testing.

- **ScanNet** [41] is a large-scale indoor point cloud dataset containing 1603 scans at 2 cm geometry precision. We use

1503 scans for training and 100 scans for testing following the common conditions.

- **KITTI** [37] or SemanticKITTI is a large-scale LiDAR dataset used for semantic scene understanding. It contains 22 sequences, a total of 43,552 frames of outdoor scenes collected using the Velodyne HDL-64E LiDAR sensor. There are around 120k points on average per frame. These raw floating-point coordinates are quantized to a 1mm scale.
- **Ford** [37] is another outdoor LiDAR dataset used in MPEG, which contains three 1500-frame sequences. Following the common test condition [42], we use the first sequence for training and the other two for testing. Experiments are examined using original samples at 1mm geometry precision.

3) Dynamic Coding of Object Point Cloud Sequences: We mainly evaluate the dynamic coding for object point cloud sequences for emerging holographic streaming applications. We select two sequences from 8iVFB and Owlii for testing, i.e., *soldier*, *basketball_player*, and the other six sequences for training, i.e., *longdress*, *loot*, *redandblack*, *dancer*, *model*, *erercise*.

B. Evaluation Metrics

We strictly follow the common practice of measuring the rate dissipated for each point (i.e., bits per point, bpp) and distortion in terms of PSNR in dB of the attribute channel (Y or YUV). We use the "pc_error" tool provided by MPEG to compute the distortion. For the overall quality assessment considering superimposed distortion of lossily coded geometry & attribute, we choose the popular PCQM [2] measurement.

- In lossless mode, Compression Ratio (CR) gain measured by bits per point (bpp) is used.
- In lossy mode, the Rate-Distortion (R-D) gain is derived using the well-known Bjøntegaard Delta Bit Rate (BD-BR) [43].

We also evaluate the complexity using encoding/decoding runtime and model size. As most works used in comparative studies are just algorithm prototypes, they are implemented using different languages (e.g., C++/C of standard G-PCC/V-PCC codecs vs. Python of learned solutions), and run on various hardware platforms (e.g., CPU vs. GPU, or even different types of GPUs). The runtime and model size are just used as an intuitive reference to present a general idea of algorithm complexity.

C. Model Training

In training, we use cross-entropy to measure the attribute bitrate in lossless coding, as well as the bitrate of latent features in lossy coding. In the meantime, we use Mean Square Error (MSE) loss between the original and reconstructed attribute for distortion measurement in lossy coding.

Since the multiscale representation framework is used in *Unicorn*, the total loss is the sum of losses at all scales (and sub-scales), and the corresponding model weights are determined in an end-to-end manner.

As for lossless coding mode, its loss function is:

$$L_{lossless} = \sum_{s=1}^{L}\sum_{c=1}^{C}\sum_{g=1}^{G}\left(\sum_{i=1}^{N^{(s,g,c)}} -\log_2\left(p(x_i^{(s,g,c)})\right)\right), \tag{11}$$

where L stands for the total number of scales. $C=3$ denotes three-channel YCoCg color attribute, while $C=1$ denotes single-channel reflectance. G is set as 8. $N^{(s,g,c)}$ is the number of points for color component c in group g at scale s; and p is estimated using (5).

On the other hand, the loss function for lossy coding scales is

$$L_{lossy} = \sum_{s=m+1}^{L}\left(\sum\nolimits_{i=1}^{N^{(s)}} -\log_2\left(p(\hat{f}_i^{(s)})\right)\right) + \lambda\cdot\sum_{s=m+1}^{L}\left(\sum\nolimits_{i=1}^{N^{(s)}}\|x_i-\hat{x}_i\|_2^2\right), \tag{12}$$

where s is the scale index, $N^{(s)}$ is the number of points at the scale s, and p is estimated using (9). λ determines the R-D trade-off. During the training, we randomly set λ between 2^8 and 2^{15} for object point clouds and ScanNet samples, and between 2^3 and 2^9 for LiDAR samples to derive a single model supporting variable bitrates.

The learning rate is set as 0.00005in training. The training iteration executes around 200 epochs with a batch size of 4, lasting around two days in our environment. Adam is used as the optimizer [44].

V. Performance Evaluation

Extensive experiments are performed to understand the efficiency of attribute compression using the proposed *Unicorn*.

A. Comparison With Standard Codecs

G-PCC is mainly taken as the anchor for comparison. Evaluation strictly follows the standardization CTC [37].

Static Coding uses the TMC13v21 of G-PCC, to obtain the performance results for comparison.

- In lossy mode, RAHT is used in G-PCC as it provides much better efficiency than other techniques across almost all categories [45].
- In lossless mode, Predicting Transform is applied in G-PCC for its superior performance.

Attribute compression efficiency is reported with losslessly compressed geometry, while overall performance considering the geometry and attribute jointly is measured for lossily compressed geometry and attribute.

Dynamic Coding employs GeSTMv3 [46], which is under extensive study in MPEG for compressing dynamic object sequences.

V-PCC is also tested using TMC2v18 reference software [15]. As V-PCC is more recommended for networked immersive applications [47], we mainly consider the lossy compression of both geometry and attribute information when comparing it with the proposed *Unicorn*.

Next, we discuss the compression of point cloud attributes with losslessly processed geometry first.

TABLE III
QUANTITATIVE GAINS TO THE G-PCC FOR *STATIC CODING*

PCs	Lossless			Lossy	
	G-PCC bpp	*Unicorn* bpp	CR Gain	BD-BR Gain Y	BD-BR Gain YUV
8iVFB 10-bit					
longdress	11.554	10.037	−13.1%	−24.4%	−17.2%
loot	6.061	5.362	−11.5%	−19.7%	−22.0%
red&black	9.204	8.096	−12.0%	−24.3%	−8.5%
soldier	6.931	5.837	−15.8%	−25.2%	−28.5%
Average	8.438	7.333	**−13.1%**	**−23.4%**	**−19.1%**
Owlii 10-bit					
player	8.033	7.385	−8.8%	−33.3%	−32.2%
dancer	8.100	7.336	−10.4%	−31.0%	−27.9%
model	8.288	7.272	−14.0%	−27.8%	−27.9%
exercise	8.607	8.135	−5.8%	−24.2%	−22.6%
Average	8.257	7.532	**−8.8%**	**−29.1%**	**−27.7%**
CTC Samples 11,12-bit					
player	7.674	7.140	−7.0%	−16.3%	−14.6%
dancer	7.744	7.156	−7.6%	−15.4%	−12.3%
thaidancer	7.437	6.466	−13.1%	−22.5%	−17.7%
longdress	9.647	8.862	−8.1%	−20.5%	−12.7%
loot	4.965	4.797	−3.4%	−10.6%	−9.7%
red&black	7.639	7.253	−5.1%	−8.1%	+7.6%
soldier	6.132	5.547	−9.5%	−18.7%	−17.4%
boxer	5.920	5.706	−3.6%	−2.6%	−0.1%
Facade	11.605	11.246	−3.1%	−23.5%	−22.4%
House	10.278	10.180	−1.0%	+3.6%	+3.4%
Shiva	15.351	15.210	−0.9%	+8.2%	+9.1%
Staue	13.457	12.821	−4.7%	−6.8%	−4.6%
Average	8.987	8.532	**−5.6%**	**−11.1%**	**−7.6%**
Scenes					
ScanNet	13.555	12.256	**−9.6%**	**−21.6%**	**−22.1%**
KITTI	4.764	4.364	**−8.4%**	**−4.8%**	-
Ford	5.421	5.130	**−5.4%**	**−5.3%**	-

The bold values are meant to highlight the average results for each category.

TABLE IV
RUNTIME COMPARISON WITH G-PCC FOR *STATIC CODING*

Runtime(s)	Lossless				Lossy			
	G-PCC		*Unicorn*		G-PCC		*Unicorn*	
	enc	dec	enc	dec	enc	dec	enc	dec
8iVFB	**10.4**	**10.2**	14.1	14.6	6.9	6.0	**2.9**	**1.6**
Owlii	**8.1**	**7.9**	11.5	12.2	5.4	4.7	**2.3**	**1.2**
CTC Samples	**49.0**	**48.0**	55.7	55.2	**17.2**	15.3	19.0	**14.9**
ScanNet	**2.2**	**2.2**	5.0	5.3	1.3	1.2	**1.0**	**0.5**
KITTI	**1.3**	**1.3**	6.7	6.7	**1.3**	**1.2**	3.6	2.0
Ford	**1.0**	**1.0**	4.9	4.9	**1.2**	**1.3**	3.3	1.8

The bold values are meant to highlight the best performing results in the comparison.

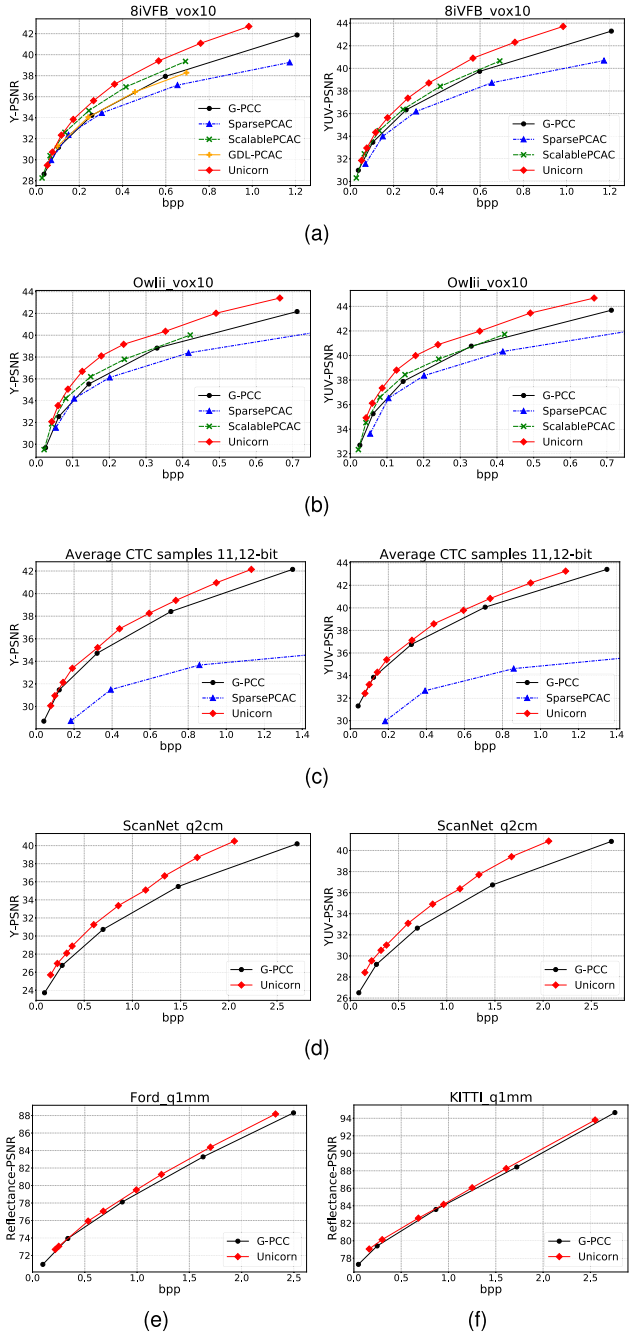


Fig. 8. Lossy R-D comparison of *Unicorn* to G-PCC and learning-based methods on static attribute coding.

1) Static Attribute Coding: Lossless Mode:

- *Quantitative CR Gains:* In Table III, *Unicorn* offers significant gains over G-PCC across a variety of datasets: For object point clouds, we provide 13.1% and 8.8% bitrate reduction to G-PCC for 8iVFB, Owlii, respectively, and an average 5.6% gain on CTC samples having higher geometry precision and more diverse geometric densities. While for scene point clouds, we achieve 9.6%, 8.4%, and 5.4% gains on ScanNet, KITTI, and Ford datasets, respectively. These results suggest *Unicorn* is well generalizing itself on various object and scene point clouds with different geometric and texture characteristics.
- *Runtime:* We also measure the lossless coding runtime of G-PCC and our method in Table IV. Quantitatively, our runtime is on the same order of magnitude as G-PCC: for object point clouds, the runtime is close, e.g., 14s vs. 10s on 8iVFB; on the other hand, for sparse KITTI and Ford datasets, G-PCC is around 5× faster than *Unicorn*, because of its dedicated tools engineered to accelerate the process of isolated points [48], while *Unicorn* uniformly treats all points.

Lossy Mode:

- *Quantitative BD-BR Gains:* As summarized in Table III using BD-BR metric and visualized Fig. 8 using R-D curves, the proposed *Unicorn* significantly outperforms

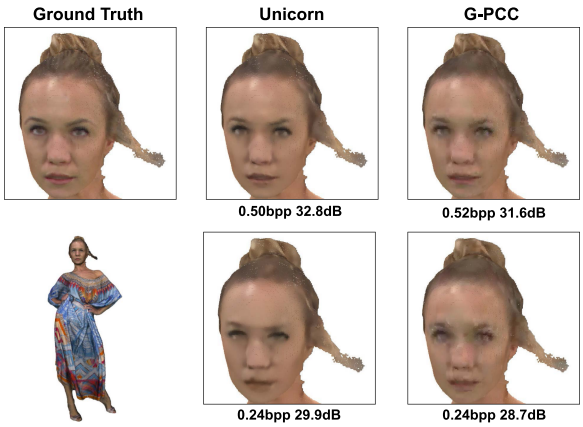


Fig. 9. Qualitative results of reconstructed point clouds of *Unicorn* and G-PCC with *lossy attribute coding and lossless geometry coding*. The bitrate and Y-PSNR are provided.

TABLE V
PERFORMANCE GAINS TO THE G-PCC AND RUNTIME COMPARISON FOR *DYNAMIC ATTRIBUTE CODING*

Lossless PCAC		G-PCC		*Unicorn*	
		Static	**Dynamic**	Static	**Dynamic**
bpp	soldier	7.18	-	5.96	5.45
	ball	8.09	-	7.38	7.29
	average	7.64	-	6.67	**6.37**
	CR Gain	anchor	-	–12.7%	**–16.6%**
Time (s)	**Enc**	10.9	-	10.6	13.2
	Dec	10.6	-	10.3	12.9

Lossy PCAC		G-PCC		*Unicorn*	
		Static	**Dynamic**	Static	**Dynamic**
BD-BR Y	soldier	-	–44.2%	–26.1%	–45.4%
	ball	-	–24.9%	–36.9%	–41.0%
	average	anchor	–34.6%	–31.5%	**–43.2%**
BD-BR YUV	soldier	-	–45.2%	–25.3%	–44.6%
	ball	-	–24.8%	–36.0%	–42.5%
	average	anchor	–35.0%	–30.7%	**–43.6%**
Time (s)	**Enc**	5.9	8.9	**3.2**	4.9
	Dec	5.2	9.4	**1.7**	3.2

G-PCC across a wide bitrate range for various point cloud types.

For 8iVFB, Owlii, and ScanNet datasets, *Unicorn* offers about 20% – 30% BD-BR improvement over G-PCC when compressing the RGB colors.

For the CTC samples, *Unicorn* provides 11.1% and 7.6% BD-BR gains respectively over G-PCC using Y and YUV PSNR measures. Unicorn experiences some small performance losses to G-PCC on point cloud "Shiva" and "House". This is because some samples like "Shiva" present a much sparser and noisier appearance than others. Though the training dataset RWTT comprises solid samples with smooth surfaces, greatly differing from "Shiva", *Unicorn* consistently shows its strong generalization capacity across various testing content. Including similar training samples or properly augmenting noises can help improve *Unicorn*'s performance on these noisy samples, which is deferred as our future study.

Regarding the reflectance attribute of scant LiDAR datasets, we achieve 4.8% and 5.3% gains over G-PCC on KITTI and Ford datasets, respectively.

- *Runtime:* Our lossy coding has a comparable runtime at the same order of magnitude as G-PCC. As presented in Table IV, for object point clouds, including 10-bit 8iVFB, Owlii datasets, our encoding and decoding runtime measures report approximately $2\times$ – $4\times$ speedup to G-PCC. We have similar runtime to G-PCC on 11-bit and 12-bit CTC samples, this is due to their large number of points, i.e., $> 3\times$ than 8iVFB. Thus we have to chunk and process each part serially, constrained by our GPU memory. While for scant KITTI and Ford datasets, G-PCC is faster, but the runtime measures for both methods are affordable at the second level.
- *Qualitative Visualization:* As illustrated in Fig. 9, we have compared the reconstructions of lossy attribute coding with lossless geometry. The distinction in distortion can be clearly observed on the human face: our reconstructions exhibit a smooth texture that is visually pleasing. In contrast, the reconstructions of G-PCC are filled with unsettling noise. Overall, the proposed *Unicorn* can preserve more details than G-PCC, reflecting more than 1dB higher PSNR.

2) Dynamic Attribute Coding: Table V reports the performance of dynamic attribute coding in both lossless and lossy scenarios. G-PCC's static coding mode is the anchor for quantitative improvement measures.

In lossless mode, *Unicorn*'s static and dynamic coding modes provide 12.7% and 16.6% CR gains against the anchor respectively. Since G-PCC's dynamic coding is still in its early development phase, and the current focus is lossy compression, its lossless compression unfortunately presents an inferior performance compared with the anchor. Improving the lossless compression mode in G-PCC's dynamic coding is a tentative topic of standard committee. Here, we chose not to report the interim results here.

In lossy mode, *Unicorn* achieves more than 30% and 43% BD-BR gains to the anchor setting for respective static and dynamic coding modes, while G-PCC's dynamic coding brings about 35% BD-BR improvement, which is inferior to *Unicorn*'s dynamic coding mode with more than 8% behind. R-D plots in Fig. 10 further confirm the superior efficiency of *Unicorn* across a wide range of bitrates.

As shown in the runtime comparison of Table V, the dynamic coding of *Unicorn* is lightweight, having a comparable encoding/decoding speed to the anchor G-PCC. Compared to the static coding mode of *Unicorn*, the dynamic coding mode does not increase the runtime significantly, which owes to the simple yet efficient Extractor and Warper used.

The proposed dynamic coding of *Unicorn* notably improves the static coding performance in both lossless and lossy modes. This is because compared to the static coding in which only the spatial prior can be leveraged and fed into APUs, the proposed spatiotemporal conditional coding in dynamic mode can exploit more prior information from the reference frames. Furthermore, the gains reported for the "soldier" sequence are significantly higher than that for the "basketball_layer" sequence. This is

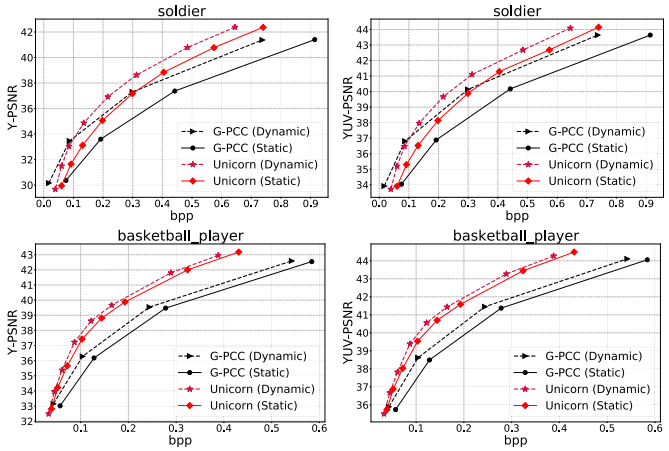


Fig. 10. R-D comparison for dynamic attribute coding in lossy mode.

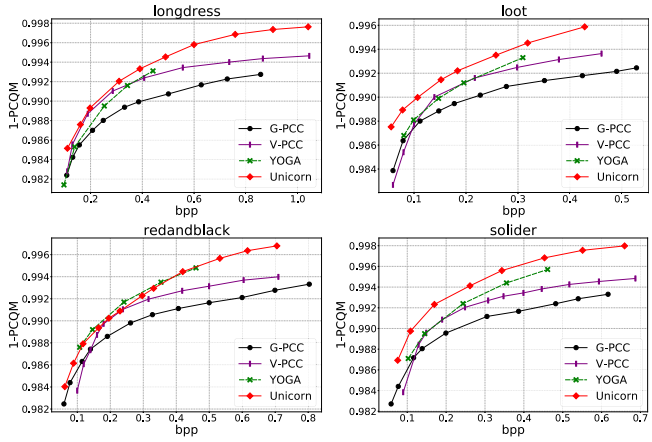


Fig. 11. R-D comparison for lossy compression of geometry & attribute.

because the latter sequence has more intense motion activities across temporal frames, which may not be sufficiently captured by the simple target convolution in this work. As for the next steps, devising a better 3D motion model is an interesting topic to pursue potential improvement.

3) Lossy Compression of Geometry & Attribute: Previous discussions assume losslessly compressed geometry, for which only attribute compression efficiency is evaluated. In practice, especially for sustaining point cloud-based applications across the Internet, lossy compression of both geometry and attribute components is highly desired. We use widely recognized PCQM [2] to evaluate superimposed impairments from compressed geometry and attribute components as a distortion index. Such a PCQM metric calculates and weighs geometric and color-based features, including curvature structure, brightness, etc., to quantify the total distortion.

Given a bitrate budget, we traverse all possible geometry and attribute coding combinations and then select the one producing the best PCQM measure. As *Unicorn* supports the variance-rate coding for both geometry and attribute, such a traversal is straightforward and effective. For a fair comparison, we apply the same traversal strategy to the existing three baselines, G-PCC, V-PCC, and YOGA [24]. Both G-PCC and V-PCC are standard-compliant solutions, while YOGA is a learning-based approach discussed in Section V-B-3.

Instead of time-consuming traversal, devising an intelligent algorithm to perform the bitrate allocation between compressed geometry and attribute information promptly and automatically is highly desired for live applications, which is deferred as our future study.

Quantitative Gains: The Rate-PCQM curves in Fig. 11 and BD-BR gains in Table VI show that *Unicorn* significantly outperforms G-PCC and V-PCC, i.e., averaged $>40\%$ gain over G-PCC and $>25\%$ gain over V-PCC.

Qualitative Visualization: Point cloud reconstructions compressed using G-PCC and *Unicorn* in the lossy mode of both geometry and attribute are visualized in Fig. 12 for two different bitrate targets. Results of other bitrates have similar behaviors. As seen, G-PCC reconstructions lose geometry and attribute details significantly, exhibiting visually unpleasant blurring and blocky artifacts. One reason is that G-PCC's geometry coding greatly suffers from point vanishing and displacement, significantly deteriorating the overall performance. In contrast, *Unicorn* reconstructions provide much smoother and fine-grained surfaces, which reveals the superiority of *Unicorn* in retaining critical details of both geometry and attribute information.

The quantitative and qualitative results demonstrate the clear performance lead of *Unicorn* over standard solutions, promising its potential as a candidate for next-generation, AI-based point cloud compression standard.

TABLE VI
QUANTITATIVE GAINS FOR LOSSY COMPRESSION OF GEOMETRY & ATTRIBUTE

PCs	G-PCC	V-PCC	YOGA [24]
longdress	–39.9%	–13.5%	–16.6%
loot	–51.8%	–31.8%	–29.3%
red&black	–32.1%	–20.7%	+6.5%
soldier	–51.2%	–36.2%	–26.8%
Average	**–43.8%**	**–25.6%**	**–16.6%**

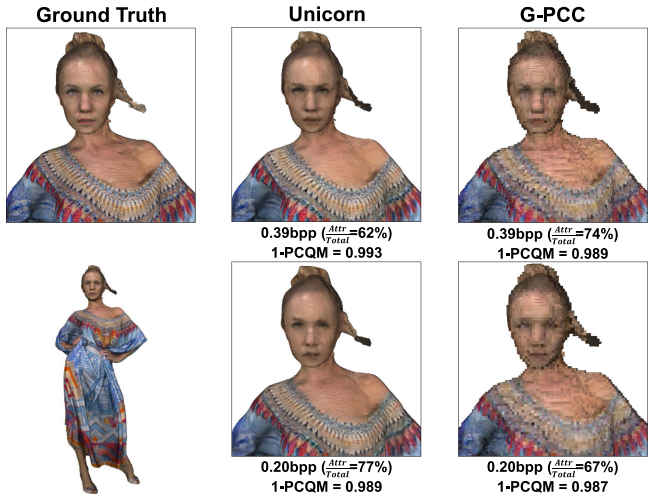


Fig. 12. Qualitative visualization of point cloud reconstructions compressed using *Unicorn* and G-PCC with *lossy compression mode of both geometry & attribute*. The total bitrate (as a target), bitrate proportion of compressed attributes, and 1-PCQM are also provided.

TABLE VII
QUANTITATIVE COMPARISON FOR LOSSLESS ATTRIBUTE CODING

Lossless PCAC Methods		G-PCC	*Unicorn*	CNeT [19]	MNeT [20]
bpp	red&black	9.20	8.07	7.21	12.33
	loot	6.09	5.18	4.65	8.38
	Average	7.65	6.63	**5.93**	10.31
	CR Gain	anchor	−13.3%	**−22.5%**	+34.8%
Time	Enc	10.4s	14.1s	25s	**3.0s**
	Dec	**10.2s**	14.6s	10min	-
Model Size		**0**	200MB	240MB	12MB

The runtime of CNeT and MNeT are quoted from their papers [19], [20], the numbers are only used for reference.

The bold values are meant to highlight the best performing results in the comparison.

TABLE VIII
QUANTITATIVE COMPARISON WITH LEARNED LOSSY ATTRIBUTE COMPRESSION METHODS

Lossy PCAC Methods		*Unicorn*	Sparse PCAC [3]	Scalable PCAC [4]	GDL PCAC [5]
BD-BR (Y)	longdress	−24.4%	+10.2%	−14.7%	+3.1%
	loot	−19.7%	+55.8%	−5.2%	+4.3%
	red&black	−24.3%	+19.7%	−13.5%	+7.3%
	soldier	−25.2%	+19.3%	−11.0%	+6.0%
	Average	**−23.4%**	+26.3%	−11.1%	+5.2%
BD-BR (YUV)	longdress	−17.2%	22.3%	−0.7%	-
	loot	−22.0%	70.8%	−2.5%	-
	red&black	−8.5%	51.2%	3.5%	-
	soldier	−28.5%	32.7%	−7.4%	-
	Average	**−19.1%**	+44.3%	−1.8%	-
Time	Enc	**2.9s**	1.5min	1.5min	8min
	Dec	**1.6s**	9min	9min	6min
Model Size		**65MB×1**	76MB×N	165MB×N	-

Unicorn, SparsePCAC, and ScalablePCAC used the same training dataset (i.e., RWTT) and test device (i.e., NVIDIA RTX 3090 GPU). While GDL-PCAC used 8iVFB+MVUB dataset for training and tested on NVIDIA RTX 2080 Ti GPU as reported in the paper [5]. N implies the number of bitrate points. Anchor is G-PCC.

B. Comparisons With Alternative Learned Methods

We also compare the attribute coder in *Unicorn* with multiple recently emerged learning-based methods to showcase its efficiency. These learned solutions are categorized below:

1) Lossless Attribute Coding: CNeT [19] and its fast version MNeT [20].
2) Lossy Attribute Coding:
   - End-to-end solutions: SparsePCAC [3] and GDL-PCAC [5].
   - Layered Enhancement: ScalablePCAC [4].
3) Lossy Coding of Geometry & Attribute: a layered solution with G-PCC at the base layer and neural autoencoder at the enhancement layer – YOGA [24].

Note that lossless geometry is assumed for options 1) and 2) above. For fairness, we maintain the same training dataset when comparing with other methods. For example, we use the RWTT dataset to retrain SparsePCAC and ScalablePCAC for comparison. While for lossless coding methods CNeT and MNeT, we retrain *Unicorn* using their training datasets and directly quote their results in the papers for comparison.

1) Lossless Attribute Coding: As shown in Table VII, we compare CR gains to the G-PCC anchor and collect the runtime as the computational complexity. CNeT currently provides the best compression ratios at the expense of a very long decoding time, i.e., around 10 minutes, due to the sequential processing of the underlying autoregressive context model. It also has the largest model size, i.e., 240 MB. MNeT significantly speeds up the runtime using the multiscale parallel structure, but its performance suffers, i.e., >34% loss against G-PCC, which is mainly due to the lack of exploiting cross-scale correlations. In contrast, *Unicorn* achieves $>13\%$ bitrate reduction over G-PCC with a similar runtime, i.e., 14s (e.g., $>40\times$ speed up of CNeT's decoding). *Unicorn*'s model size is 200 MB. *Unicron*'s better trade-off between compression ratios and complexity justifies its application potential in practice.

2) Lossy Attribute Coding: We compare *Unicorn* with other learning-based lossy PCAC methods in terms of R-D performance and complexity (runtime and model size), as shown in Table VIII and Fig. 8. The experimental comparison demonstrates the superiority of *Unicorn*, e.g., state-of-the-art gains over G-PCC anchor with $>20\%$ BD-BR improvement, the least runtime using a few seconds, and a single small-size model for variable-rate coding, i.e., only 65MB, which is the smallest of these methods.

On the contrary, end-to-end solutions, for example,

- SparsePCAC [3] underperforms G-PCC anchor by 26% (44%) BD-BR loss when measuring Y (YUV) distortion, especially at higher bitrates (see Fig. 8(a)). The inferior performance of SparsePCAC suggests that simply devising stacked (sparse) convolutions as a nonlinear transform can not sufficiently exploit attribute correlations in a local neighborhood. Meanwhile, it has a long encoding and decoding time (i.e., 1.5 minutes and 9 minutes) because of using an autoregressive context model. It requires rate-specific models (i.e., 76 MB$\times N$, where N indicates the number of bitrate points).
- GDL-PCAC [5], instead, proposed the graph dictionary learning with ρ -Laplaican embedding to learn a better transform for lossy attribute compression. It outperforms GFT [49], RAHT [14], GSR [50], and SparsePCAC, but still shows 5.2% loss to the anchor.[3] Its long runtime is mainly due to the numerical iterations for non-convex optimization.

On the other hand, neural enhancements to G-PCC, such as layered augmentation, recently demonstrated encouraging gains to the anchor, although inferior to *Unicorn*.

- ScalablePCAC [4] is a layered compressor that utilizes G-PCC to encode the base layer and a neural model (like SparsePCAC) to encode the enhancement layer, taking advantage of both rules-based and learning-based techniques for better R-D performance. It achieves 11% (2%) BD-BR reduction when measuring Y (YUV) distortion against G-PCC. However, it requires a long encoding/decoding runtime as it also employs the autoregressive context model for entropy coding and uses large-size and rate-specific models i.e., 165 MB$\times N$, where N means the number of bitrate points.

[3]The positive gain of GDL-PCAC to G-PCC (TMC13v14) was reported in its paper. Since TMC13v21 used in this work has improved TMC13v14 noticebly [45], the advantage of GDL-PCAC is diminished.

TABLE IX
ON MULTISTAGE CONDITIONAL CODING FOR LOSSLESS ATTRIBUTE COMPRESSION

Lossless PCAC	G-PCC	CS	CS+CG	CS+CG+CC
bpp	8.44	9.11	7.57	**7.33**
Gain	anchor	7.9%	–10.3%	**–13.2%**
enctime(s)	14.4	**5.3**	9.7	14.1
dectime(s)	13.2	**4.7**	9.3	14.6

CS: cross-scale; CG: cross-group; CC: cross-color.

The bold values are meant to highlight the best performing results in the comparison.

3) Lossy Compression of Geometry & Attribute: YOGA [24] is a recently-emerged PCC method that considers both geometry & attribute compression in lossy mode. It is a layered solution, as introduced in the related work. Compared with *Unicorn* that is fully built up with neural networks, YOGA has merit for backward compatibility with G-PCC since it directly uses G-PCC to encode thumbnail point clouds at the base layer.

To ensure a fair comparison, we share the same training and test conditions with YOGA in the experiments. As shown in Table VI and Fig. 11, *Unicorn* surpasses YOGA by an average 16% BD-BR gain with a broader bitrate coverage. The improvement of *Unicorn* to YOGA is attributed to the cross-scale processing mechanism (e.g., probability approximation or predictive/progressive reconstruction).

C. Discussion

Extensive comparative studies in previous sections report the outstanding performance of *Unicorn*. It is also worth pointing out that *Unicorn* just uses simple sparse convolutional layers and does not require specific tricks to engineer the neural network unit. As a result, its outstanding compression efficiency is mainly attributed to the universal multiscale conditional coding mechanism, using which neighborhood correlations can be exhaustively exploited at scale level or sub-scale level.

VI. ABLATION STUDIES

Unicorn's capacity is further examined in this section.

A. Multistage Conditional Coding

We study the impact of multistage conditional coding, which is devised to exploit sub-scale correlations. Such a sub-scale organization is facilitated by partitioning the current scale data into multiple groups, color channels, etc.

We examine the performance-complexity trade-off of cross-scale (CS), cross-group (CG), and cross-color (CC) prediction modules in lossless coding, as shown in Table IX. The coding gain over G-PCC increases from the baseline using only CS prediction with 7.9% loss to the case using both CS and CG predictions with 10.3% positive gain. When the CC prediction is further included, the bitrate reduction is improved to 13.2%. The complexity measured using encoding and decoding runtime increases when involving more sub-scale predictions. Nevertheless, it is still comparable to G-PCC at the second level. We apply the option of "CS+CG+CC" in this work for lossless attribute coding.

Note that cross-scale and cross-group predictions are also used in geometry coding. Cross-color prediction is applicable for multi-channel attributes like YCoCg.

Multistage conditional coding can also be applied in lossy attribute coding, such as dimension-wise transformation RAHT [14] and checkerboard context models in image coding solutions [51], [52]. We leave these interesting topics for future study.

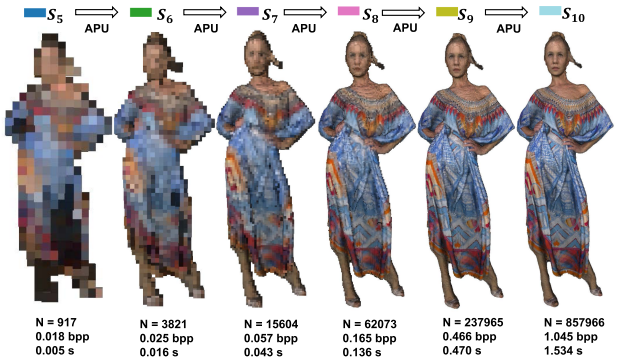


Fig. 13. Visualization of progressive decoding/reconstruction. A total number of points N, bitrate in bpp, and decoding time in second (s) are supplemented for each scale.

B. Progressive Decoding/Reconstruction

Unicorn's lossy attribute coder comprises a lossless phase from scale s_1 to s_m and a lossy phase from s_{m+1} to s_L. As for the lossy part, transform-domain conditional coding of attribute residual is repeated, which, as a result, provides the residual refinement scale by scale and thus offers the coarse-to-fine representation capability.

As shown in Fig. 13, we take "longdress" as an example to visualize the progressive decoding/reconstruction through residual refinement scale by scale. We can observe the progressive restoration of geometry and texture details from s_5 to s_{10} ($m = 5$ in this example). In each scale, the incremental residuals are augmented to provide a better reconstruction. We also supplement the total number of points, bitrate, and the decoding time at each scale underneath the "longdress" plot as in Fig. 13.

Such a progressive decoding functionality is practically beneficial for network transmission with varying bandwidth or resource-constraint computing devices, with which we can choose to drop the packets at higher scales accordingly.

C. Lossless Scales in Lossy Coding

Unicorn's lossy coding mode is easily facilitated using a two-phase approach, where the first phase applies the lossless coding from the first (lowest) scale s_1 to s_m, and then lossy coding is devised from s_{m+1} to the highest scale s_L. The highest scale

TABLE X
PERFORMANCE IMPACT OF USING DIFFERENT PROBABILITY DISTRIBUTIONS IN LOSSLESS ATTRIBUTE CODING

bpp	Laplacian	Gaussian	Gain
longdress	10.037	9.980	–0.5%
loot	5.362	5.302	–1.1%
red&black	8.096	8.069	–0.3%
soldier	5.837	5.794	–0.7%
Average	**7.333**	**7.286**	**–0.6%**

is often set the same as the input resolution. Such a strategy is uniformly used in geometry and attribute coders. Whereas,

- In geometry coder as detailed in Part I [1], adapting m controls the geometry bitrate; in contrast,
- m in attribute coder is often fixed since AQL is used to control bitrates. For example, $m = 5$ for object point clouds used in this work.

D. Impact of Using Different Probability Distributions

This section compares the impact of using Laplacian or Gaussian distributions in the CPA model for lossless attribute coding. As shown in Table X, the compression performance variations are negligible, suggesting either of them can be used. This work uses the Laplacian distribution following SparsePCGC [53].

VII. CONCLUSION AND FUTURE WORK

Unicorn's attribute coder demonstrates state-of-the-art efficiency in compressing point cloud attributes (assuming lossless geometry prior), regardless of lossy or lossless, static or dynamic, dense or sparse input, etc, at an affordable cost of complexity.

Its high efficiency owes to the universal multiscale conditional coding, by which we can thoroughly exploit scale- or sub-scale correlations from spatial or spatiotemporal neighbors.

Its low computational consumption is due to using simple neural networks. A small amount of storage for its neural models is desired as it supports variable-rate coding using a single neural model.

The outstanding performance is retained when compressing the geometry and attribute information of any input source (from the solid object to scant LiDAR), significantly outperforming standard solutions like MPEG G-PCC and V-PCC.

Currently, *Unicorn* offers a new starting point for point cloud compression, which we will soon release to the public for reproducible research and welcome more interested parties to improve it. It supports

- Lossy and Lossless Coding
- Static and Dynamic Coding
- Geometry and Attribute Coding
- Variable-rate Coding
- Network-friendly Layered Coding

Users can manually choose the proper option(s) to fulfill the specific purpose.

There are multiple exciting directions to explore for next steps, including but not limited to:

- Advanced 3D motion module to handle large and complex motion, especially for LiDAR scans.
- Intelligent rate control between geometry and attribute components given a total bitrate budget.
- A quality index with better correlation with human assessments to evaluate the superimposed geometry and attribute distortions, such as colorized object point clouds used in AR/VR applications.

REFERENCES

[1] J. Wang, R. Xue, J. Li, D. Ding, Y. Lin, and Z. Ma, "A versatile point cloud compressor using universal multiscale conditional coding – part I: Geometry," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, pp. 1–18 Sep. 17, 2024, doi: 10.1109/TPAMI.2024.3462938.

[2] G. Meynet, Y. Nehmé, J. Digne, and G. Lavoué, "PCQM: A full-reference quality metric for colored 3D point clouds," in *Proc. 12th Int. Conf. Qual. Multimedia Experience*, 2020, pp. 1–6.

[3] J. Wang and Z. Ma, "Sparse tensor-based point cloud attribute compression," in *Proc. IEEE 5th Int. Conf. Multimedia Inf. Process. Retrieval*, 2022, pp. 59–64.

[4] J. Zhang, J. Wang, D. Ding, and Z. Ma, "Scalable point cloud attribute compression," *IEEE Trans. Multimedia*, early access, pp. 1–11, Nov. 09, 2023, doi: 10.1109/TMM.2023.3331584.

[5] X. Li, W. Dai, S. Li, C. Li, J. Zou, and H. Xiong, "3-D point cloud attribute compression with p-laplacian embedding graph dictionary learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 2, pp. 975–993, Feb. 2024.

[6] M. Waschbüsch, M. H. Gross, F. Eberhard, E. Lamboray, and S. Würmlin, "Progressive compression of point-sampled models," in *Proc. Eurographics Conf. Point-Based Graph.*, 2004, pp. 95–102.

[7] R. Schnabel and R. Klein, "Octree-based point-cloud compression," in *Proc. Symp. Point-Based Graph.*, 2006, pp. 111–121.

[8] Y. Huang, J. Peng, C.-C. J. Kuo, and M. Gopi, "A generic scheme for progressive point cloud coding," *IEEE Trans. Vis. Comput. Graphics*, vol. 14, no. 2, pp. 440–453, Mar./Apr. 2008.

[9] C. Zhang, D. Florencio, and C. Loop, "Point cloud attribute compression with graph transform," in *Proc. IEEE Int. Conf. Image Process.*, 2014, pp. 2066–2070.

[10] R. A. Cohen, D. Tian, and A. Vetro, "Attribute compression for sparse point clouds using graph transforms," in *Proc. IEEE Int. Conf. Image Process.*, 2016, pp. 1374–1378.

[11] Y. Shao, Z. Zhang, Z. Li, K. Fan, and G. Li, "Attribute compression of 3D point clouds using laplacian sparsity optimized graph transform," in *Proc. IEEE Int. Conf. Vis. Commun. Image Process.*, 2017, pp. 1–4.

[12] Y. Xu et al., "Predictive generalized graph fourier transform for attribute compression of dynamic point clouds," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 5, pp. 1968–1982, May 2021.

[13] F. Song, G. Li, W. Gao, and T. H. Li, "Rate-distortion optimized graph for point cloud attribute coding," *IEEE Signal Process. Lett.*, vol. 29, pp. 922–926, 2022.

[14] R. L. de Queiroz and P. A. Chou, "Compression of 3D point clouds using a region-adaptive hierarchical transform," *IEEE Trans. Image Process.*, vol. 25, no. 8, pp. 3947–3956, Aug. 2016.

[15] "V-PCC codec description," *ISO/IEC JTC 1/SC 29/WG 7 (MPEG 3DG) output document N00100*, Apr. 2022.

[16] C. Cao, M. Preda, and T. Zaharia, "3D point cloud compression: A survey," in *Proc. 24th Int. Conf. 3D Web Technol.*, 2019, pp. 1–9.

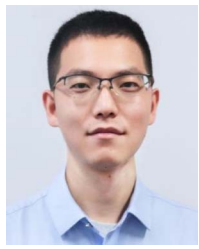
[17] E. Alexiou, K. Tung, and T. Ebrahimi, "Towards neural network approaches for point cloud compression," in *Applications of Digital Image Processing XLIII*, vol. 11510. Bellingham, WA, USA: SPIE, 2020, pp. 18–37.

[18] X. Sheng, L. Li, D. Liu, Z. Xiong, Z. Li, and F. Wu, "Deep-PCAC: An end-to-end deep lossy compression framework for point cloud attributes," *IEEE Trans. Multimedia*, vol. 24, pp. 2617–2632, 2022.

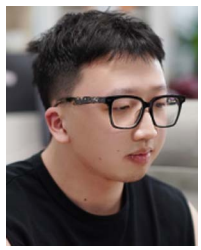
[19] D. T. Nguyen and A. Kaup, "Lossless point cloud geometry and attribute compression using a learned conditional probability model," *IEEE TCSVT*, vol. 33, pp. 4337–4348, 2023.

[20] D. T. Nguyen, K. G. Nambiar, and A. Kaup, "Deep probabilistic model for lossless scalable point cloud attribute compression," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2023, pp. 1–5.

[21] X. Sheng, L. Li, D. Liu, and Z. Xiong, "Attribute artifacts removal for geometry-based point cloud compression," *IEEE Trans. Image Process.*, vol. 31, pp. 3399–3413, 2022.
[22] J. Zhang, J. Zhang, D. Ding, and Z. Ma, "ARNet: Attribute artifacts reduction for g-pcc compressed point clouds," *Comput. Vis. Media*, 2023.
[23] J. Zhang, T. Chen, D. Ding, and Z. Ma, "G-PCC++: Enhanced geometry-based point cloud compression," in *Proc. ACM Multimedia*, Ottawa, Canada, 2023, pp. 1352–1363.
[24] J. Zhang, T. Chen, D. Ding, and Z. Ma, "YOGA: Yet another geometry-based point cloud compressor," in *Proc. ACM Multimedia*, Ottawa, Canada, 2023, pp. 9070–9081.
[25] J. Wang, D. Ding, Z. Li, and Z. Ma, "Multiscale point cloud geometry compression," in *Proc. IEEE Data Compression Conf.*, 2021, pp. 73–82.
[26] M. Quach, G. Valenzise, and F. Dufaux, "Folding-based compression of point cloud attributes," in *Proc. IEEE Int. Conf. Image Process.*, 2020, pp. 3309–3313.
[27] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the high efficiency video coding (HEVC) standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 22, no. 12, pp. 1649–1668, Dec. 2012.
[28] B. Isik et al., "LVAC: Learned volumetric attribute compression for point clouds using coordinate based networks," *Front. Signal Process.*, vol. 2, 2022, Art. no. 1008812.
[29] G. Fang, Q. Hu, H. Wang, Y. Xu, and Y. Guo, "3DAC: Learning attribute compression for point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 14 819–14 828.
[30] "G-PCC codec description v12," *ISO/IEC JTC1/SC29/WG7 (MPEG 3DG) output document N00271*, Jan. 2022.
[31] J. Wang, D. Ding, and Z. Ma, "Lossless point cloud attribute compression using cross-scale, cross-group, and cross-color prediction," in *Proc. IEEE Data Compression Conf.*, 2023, pp. 228–237.
[32] R. B. Pinheiro, J.-E. Marvie, G. Valenzise, and F. Dufaux, "NF-PCAC: Normalizing flow based point cloud attribute compression," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2023, pp. 1–5.
[33] T. Chen and Z. Ma, "Variable bitrate image compression with quality scaling factors," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2020, pp. 2163–2167.
[34] Z. Duan et al., "QARV: Quantization-aware ResNet VAE for lossy image compression," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 1, pp. 436–450, Jan. 2024.
[35] T. M. Cover and J. A. Thomas, *Elements of Information Theory (Wiley Series in Telecommunications and Signal Processing)*. Hoboken, NJ, USA: Wiley-Interscience, 2006.
[36] A. Zaghetto, D. Graziosi, and A. Tabatabai, "Dataset split for AI-PCC," *ISO/IEC JTC1/SC29/WG7 (MPEG 3DG) input document m58754*, Jan. 2022.
[37] "Common test conditions for G-PCC," *ISO/IEC JTC1/SC29/WG11 (MPEG 3DG) output document N00106*, Apr. 2021.
[38] E. d'Eon, B. Harrison, T. Myers, and P. A. Chou, "8i voxelized full bodies - a voxelized point cloud dataset," ISO/IEC JTC1/SC29 Joint WG11/WG1 (MPEG/JPEG) Input Document m40059/m74006, Jan. 2017.
[39] Y. Xu, Y. Lu, and Z. Wen, "Owlii dynamic human mesh sequence dataset," *ISO/IEC Joint MPEG/JPEG input document m41658*, Oct. 2017.
[40] A. Maggiordomo, F. Ponchio, P. Cignoni, and M. Tarini, "Real-world textured things: A repository of textured models generated with modern photo-reconstruction tools," *Comput. Aided Geometric Des.*, vol. 83, 2020, Art. no. 101943.
[41] A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nieß ner, "ScanNet: Richly-annotated 3D reconstructions of indoor scenes," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2432–2443.
[42] A. Zaghetto D. Graziosi, and A. Tabatabai, "Performance analysis of currently AI-based available solutions for PCC," *ISO/IEC JTC 1/SC 29/WG 7 (MPEG 3DG) output document N0174*, Jul. 2021.
[43] G. Bjøntegaard, "Calculation of average PSNR differences between RD-curves," *ITU-T SG 16/Q6, 13th VCEG Meeting. Document VCEG-M33*, Apr. 2001.
[44] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[45] "Guidelines to use G-PCC for achieving best compression performances," *ISO/IEC JTC1/SC29/WG7 (MPEG 3DG) output document N0598*, Apr. 2023.
[46] "Test model for geometry-based solid point cloud - GeS TM 3.0," *ISO/IEC JTC1/SC29/WG7 (MPEG 3DG) output document N00686*, Sep. 2021.
[47] ISO/IEC 23090-5 - Information technology - Coded representation of immersive media - Part 5: Visual Volumetric Video-based Coding (V3C) and Video-based Point Cloud Compression (V-PCC), 2023.
[48] C. Cao, M. Preda, V. Zakharchenko, E. S. Jang, and T. Zaharia, "Compression of sparse and dense dynamic point clouds–methods and standards," *Proc. IEEE*, vol. 109, no. 9, pp. 1537–1558, Sep. 2021.
[49] S. Gu, J. Hou, H. Zeng, and H. Yuan, "3D point cloud attribute compression via graph prediction," *IEEE Signal Process. Lett.*, vol. 27, pp. 176–180, 2020.
[50] S. Gu, J. Hou, H. Zeng, H. Yuan, and K.-K. Ma, "3D point cloud attribute compression using geometry-guided sparse representation," *IEEE Trans. Image Process.*, vol. 29, pp. 796–808, 2020.
[51] D. He, Y. Zheng, B. Sun, Y. Wang, and H. Qin, "Checkerboard context model for efficient learned image compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, pp. 14 766–14 775, 2021.
[52] M. Lu and Z. Ma, "High-efficiency lossy image coding through adaptive neighborhood information aggregation," 2022, *arXiv:2204.11448*.
[53] J. Wang, D. Ding, Z. Li, X. Feng, C. Cao, and Z. Ma, "Sparse tensor-based multiscale representation for point cloud geometry compression," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 7, pp. 9055–9071, Jul. 2023.

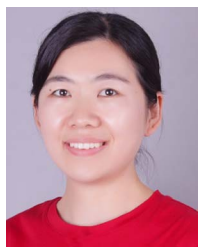


Jianqiang Wang received the BS and MS degree in electronic science and engineering from Nanjing University, Jiangsu, China in 2018 and 2021. He is currently working toward the PhD degree in Nanjing University. His research interests include point cloud compression and processing.

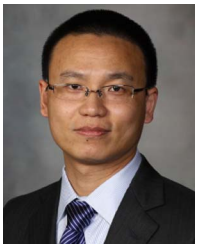


Ruixiang Xue (Graduate Student Member, IEEE) received the BS degree in electronic science and engineering from Hangzhou Dianzi University, Hangzhou, China, in 2021. He is currently working toward the PhD degree with Nanjing University. His research interests include point cloud compression and processing.

Jiaxin Li received the BS degree in optoelectronic science and engineering from the University of Electronic Science and Technology of China in 2022. She is currently working toward the PhD degree with Nanjing University. Her research interests include point cloud quality assessment.



Dandan Ding (Senior Member, IEEE) received the BEng degree (Honor.) in communication engineering from Zhejiang University, China, in 2006 and the PhD degree from Zhejiang University, in 2011. From 2007 to 2008, she was an exchange student in Microelectronic Systems Laboratory (GR-LSM) of EPFL, Switzerland. After returning to Zhejiang University, she served first as a postdoc researcher from 2011 through 2013, then as a research associate in Zhejiang University till 2015. Since 2016, she has been with the School of Information Science and Technology, Hangzhou Normal University as a faculty member with tenure track. Her research interests include artificial intelligence based image/video processing, video coding algorithm design and optimization, and point cloud processing. Currently, she is active in the development of new video coding algorithms for the new generation video coding standards, such as AOM/AV2.



Yi Lin is a cardiovascular surgeon and associate professor with the Zhongshan Hospital of Fudan University. He received medical education with Shanghai Medical School of Fudan University and then completed his Surgery, Cardiovascular and Thoracic Surgery residency with Zhongshan Hospital of Fudan University. After the clinical training, he spent one year as surgical research fellow with Mayo Clinic in Rochester followed by advanced cardiovascular surgery training at Mayo Clinic in Rochester. He returned to Zhongshan Hospital of Fudan University to join the staff in 2016 where he is currently an attending surgeon and associate professor of Cardiovascular Diseases. His primary research interest includes innovative medical imaging techniques and robot-assisted therapies in cardiovascular diseases.

Zhan Ma (Senior Member, IEEE) received the BS and MS degrees in electronic engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2004 and 2006 respectively, and the Ph.D. degree in electronic engineering from the New York University, New York, in 2011. He is now on the faculty of Electronic Science and Engineering School, Nanjing University, Nanjing, Jiangsu, China. From 2011 to 2014, he has been with Samsung Research America, Dallas TX, and Futurewei Technologies, Inc., Santa Clara, CA, respectively. His research focuses on the learning-based video coding, and smart cameras. He is a co-recipient of the 2018 PCM Best Paper Finalist, 2019 IEEE Broadcast Technology Society Best Paper Award, and 2020 IEEE MMSP Image Compression Grand Challenge Best Performing Solution.